香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

工學院
SCHOOL OF ENGINEERING

Industrial Engineering & Decision Analytics
工業工程及決策分析

Introduction to Queueing Theory

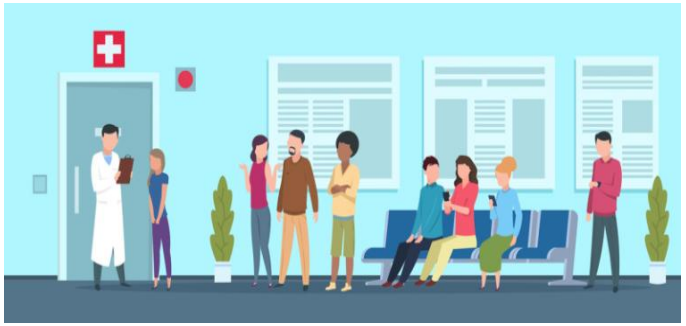
Instructor: Bo YANG

Summer, 2026

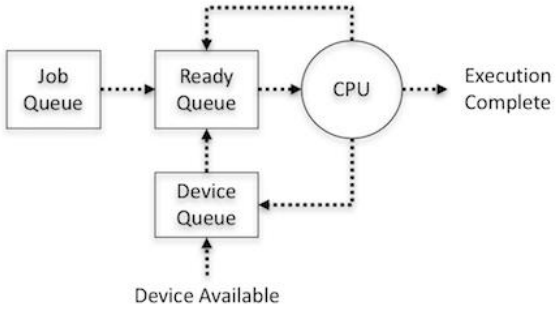
We see queues everyday

- Some conventional examples
  - Shopping malls, airports, restaurants, hospitals, banks



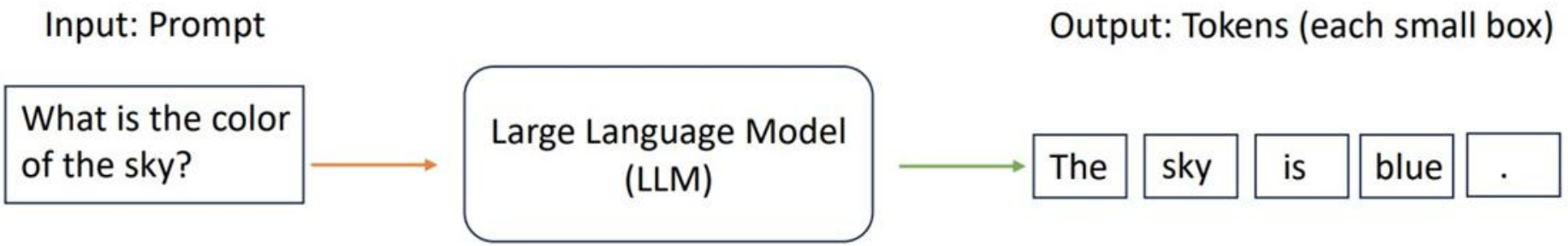


- Some unconventional examples
  - Computer system, GPU, call center



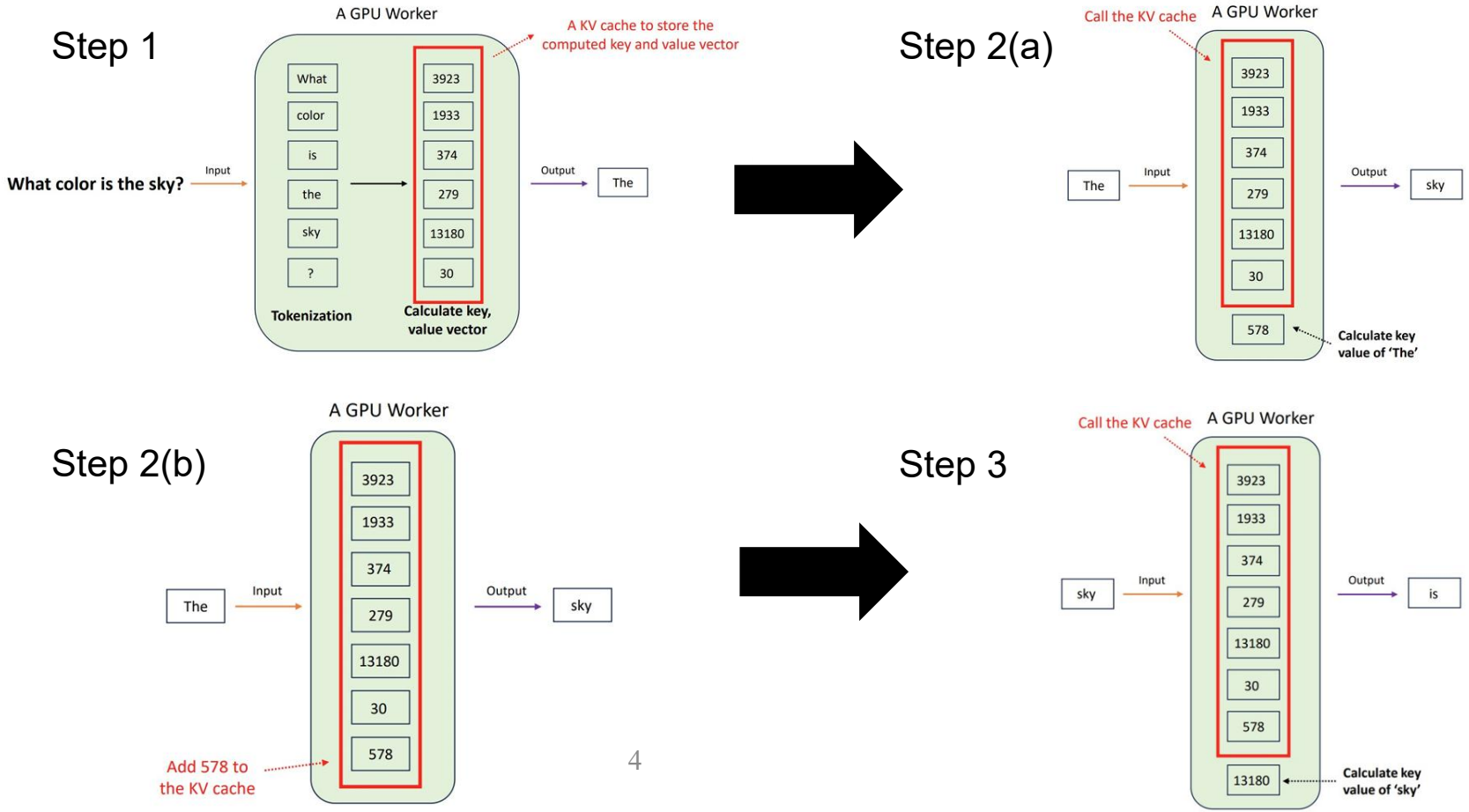
Motivation

- Consider an inference of a large language model (Lienhart, 2023)
- The input (prompt) is a sentence "What is the color of the sky?"
- The output (tokens) is also a sentence "The sky is blue."

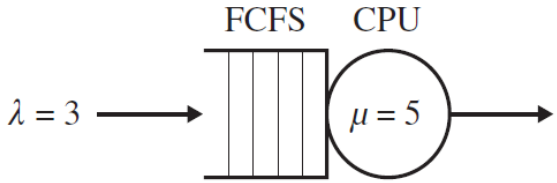


Motivation

- The LLM (with a single GPU) handles the prompt as follows



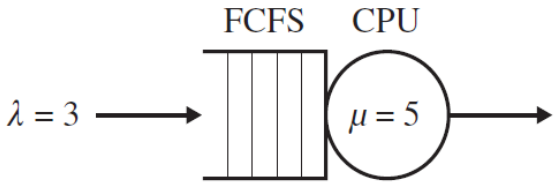
Motivation



- Consider a system consisting of a single CPU that serves a queue of jobs in **First-Come-First-Served (FCFS)** order.
- The jobs arrive according to some random process with some average arrival rate, say $\lambda = 3$ jobs per second.
- Each job has some CPU service requirement, drawn independently from some distribution of job service requirements (we can assume any distribution on the job service requirements for this example).
- Let's say that the average service rate is $\mu = 5$ **jobs per second** (i.e., each job on average requires 1/5 of a second of service).
- Note that the system is not in overload ($3 < 5$).
- Let $E[T]$ denote the mean response time of this system, where **response time** is the time from when a job arrives until it completes service, a.k.a. sojourn time.

Motivation

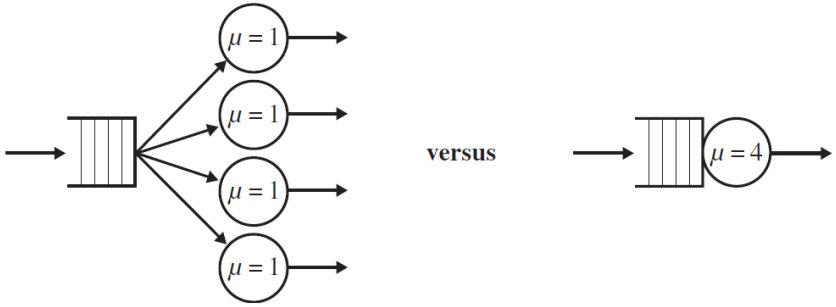
- Your boss tells you that starting tomorrow **the arrival rate will double**. You are told to buy a faster CPU to ensure that jobs experience the same mean response time, E[***T***]. **That is, customers should not notice the effect of the increased arrival rate.** By how much should you increase the CPU speed?
- (a) Double the CPU speed; (b) More than double the CPU speed; (c) Less than double the CPU speed.



- It turns out that doubling CPU speed together with doubling the arrival rate will generally result in cutting the mean response time in half! Therefore, the CPU speed does not need to double.

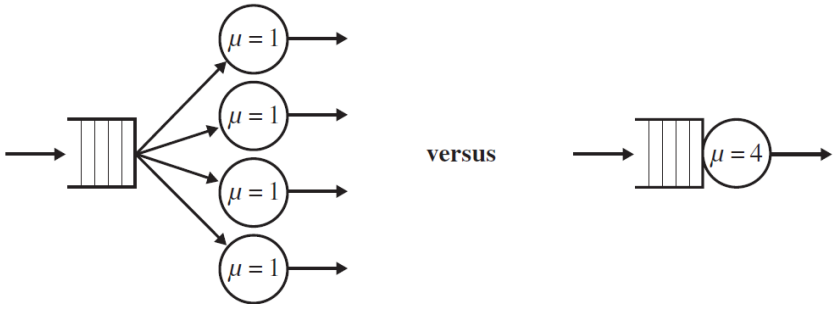
Motivation

- You are given a choice between one fast CPU of speed ***s***, or ***n*** slow CPUs each of speed ***s/n***. Your goal is to minimize mean response time. To start, assume that jobs are non-preemptible (i.e., each job must be run to completion).



Motivation

- You are given a choice between one fast CPU of speed ***s***, or ***n*** slow CPUs each of speed ***s/n***. Your goal is to minimize mean response time. To start, assume that jobs are non-preemptible (i.e., each job must be run to completion).



- It turns out that the answer depends on the variability of the job size distribution, as well as on the system load
  - Which system do you prefer when job size variability is high?
  - Which system do you prefer when load is low?
- In practice, suppose there is a fixed power budget ***P*** and a server farm consisting of ***n*** servers.
- You have to decide how much power to allocate to each server, so as to minimize overall mean response time for jobs arriving at the server farm.

Queueing theory

- Mathematical approach to the analysis of queues/wait lines

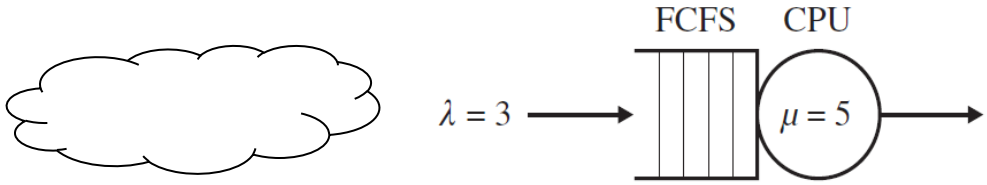
- Two goals:
  - Predicting the system performance
  - Finding a superior system design to improve performance
    - Balancing waiting cost & Service capacity cost
    - Deploying a smarter scheduling policy or different routing policy to reduce delays.

Today's plan

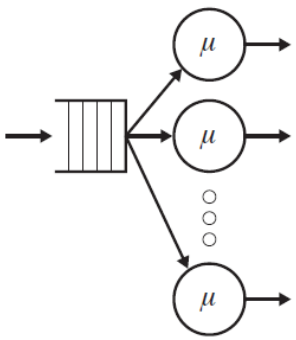
- Terminology and notation
- Little's law
- Single server queues
  - Stochastic process: Birth and death processes
- Multi server queues
  - Stochastic process: Birth and death processes

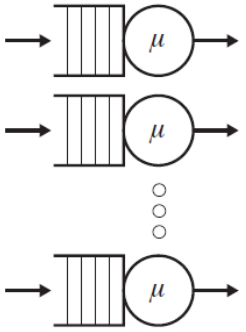
Components of a queueing model

- Queueing networks can be classified into two categories: **open networks** and closed networks.
- The calling population
  - The population from which customers/jobs originate
  - The number can be finite or **infinite** (the latter is most common)
  - Can be **homogeneous** (only one type of customers/ jobs) or heterogeneous (several different kinds of customers/jobs)
- The arrival process
  - Determines how, when and where customer/jobs arrive to the system
  - Important characteristic is the customers'/jobs' inter-arrival times
  - We always assume **Poisson arrival** in this class

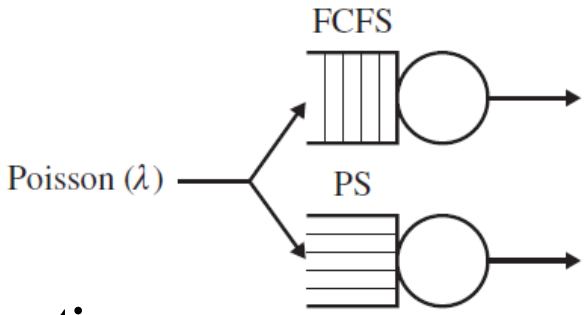


Components of a queueing model





- The queue configuration specifies
  - the number of queues
    - **Single or multiple** lines
  - the number of servers
    - **Single or multiple** servers
  - service requirement, i.e., the size of the jobs
    - Most analytical queueing models are based on the assumption of **exponentially distributed service times**
  - the order by which jobs in the queue are being served
    - **FCFS**, LCFS, PS…



Parameters and performance metrics

- **Commonly used parameters** in a queueing model
  - Average Arrival Rate $\boldsymbol{\lambda}$: the average rate at which jobs arrive to the server (e.g., 3 jobs/sec)
    - Mean Interarrival Time $\frac{\mathbf{1}}{\boldsymbol{\lambda}}$: This is the average time between successive job arrivals (e.g., $\frac{\mathbf{1}}{\boldsymbol{\lambda}}$ = 1/3 sec)
  - Average Service Rate $\boldsymbol{\mu}$: This is the average rate at which jobs are served (e.g., $\boldsymbol{\mu}$=1/E[$\boldsymbol{S}$] = 4 jobs/sec)
    - Service Requirement $\boldsymbol{S}$: The "size" of a job is denoted by the random variable $\boldsymbol{S}$
    - Mean Service Time **E[$\boldsymbol{S}$]**: This is the expected value of $\boldsymbol{S}$, namely the average time required to service a job (e.g., E[S] = 1/4 sec)

Parameters and performance metrics (Continued)

- **Commonly used metrics in a queueing model**
  - Response Time/Time in System: $\boldsymbol{T}$
  - Waiting Time: $\boldsymbol{T_Q}$
    - $E[T] = E[T_Q] + E[S]$
  - Number of Jobs in the System $\boldsymbol{N}$: This includes those jobs in the queue, plus the ones being served (if any)
  - Number of Jobs in Queue $\boldsymbol{N_Q}$

Parameters and performance metrics (Continued)

- **More metrics**
  - Throughput $\boldsymbol{X_i}$: the rate of completions at server i (e.g., jobs/sec). The throughput (X) of the system is the rate of job completions in the system
    - Let C denote the total number of jobs completed at server i during time τ. Then

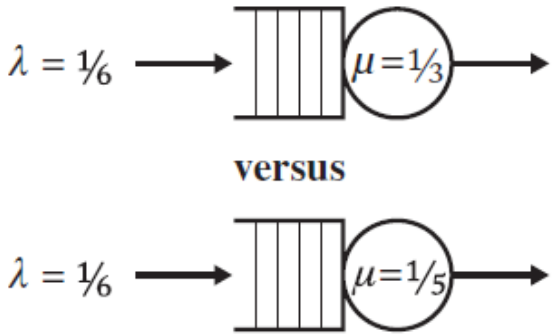
$$X_i = \frac{C}{\tau}$$

  - Utilization $\boldsymbol{\rho_i}$: the expected fraction of time the server i is busy
    - Let τ denote the length of the observation period and B denote the total time during the observation period that the server is non-idle (busy). Then

$$\rho_i = \frac{B}{\tau}.$$

Remarks on the metrics

- **Questions**
  - We always assume $\lambda < \mu$ for a single server queue. What will happen if $\lambda >= \mu$? (http://www.randhawa.us/apps/Q/)
  - Suppose the interarrival time and the service time are both deterministic, i.e., both are constants. What is T_Q? What is T?
  - How does maximizing throughput relate to minimizing response time? For example, in the following figure, which system has higher throughput?



So the throughput does not depend on the service rate whatsoever!

Remarks on the metrics

- **Questions**
  - How does X_i relate to ρ_i?

Utilization Law

$$X_i = \mu_i \cdot \rho_i \quad \Rightarrow \quad \rho_i = X_i \cdot \mathbf{E}[S] \quad \Rightarrow \quad \rho_i = \lambda_i \cdot \frac{1}{\mu_i}.$$

A question of modeling

- A few years ago, some folks at IBM were trying to model their blade server as a single-server queue. They knew the arrival rate into the server, λ, in jobs/sec. However, they were wondering how to get **E[S]**, the mean job size

- **Question:** How do you obtain **E[S]** in practice for your single-server system?

- **Attempt 1:** At first glance, you might reason that because **E[S]** is the mean time required for a job in isolation, you should just send a single job into the system and measure its response time, repeating that experiment a hundred times to get an average
  - This makes sense in theory, but does not work well in practice, because cache conditions and other factors are very different for the scenario of just a single job compared with the case when the system has been loaded for some time.
- **Attempt 2:** A better approach is to recall that **E[S] = 1/μ**, so it suffices to think about the service rate of the server in jobs/second. To get μ, assuming an open system, we can make λ higher and higher, which will increase the completion rate, until the completion rate levels off at some value, which will be rate μ

Little's Law

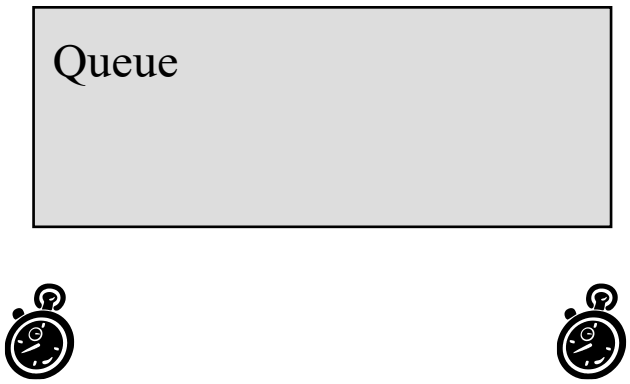
- Little's Law is probably the single most famous queueing theory result

- For any stationary open system, we have that

$$E[N] = \lambda E[T],$$

where **E[*N*]** is the expected number of jobs in the system; λ is the average arrival rate into the system; and **E[*T*]** is the mean time jobs spend in the system

$$E[N] = \lambda \times E[T]$$



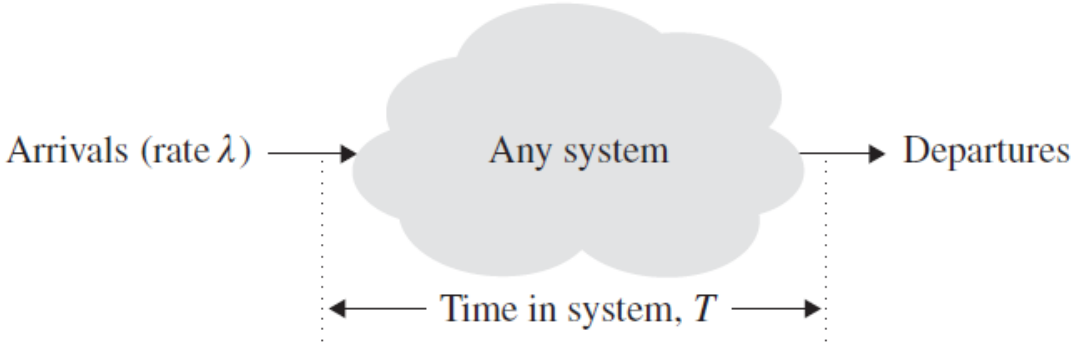
Little's Law

- Little's Law is probably the single most famous queueing theory result

- For any stationary open system, we have that

$$E[N] = \lambda E[T],$$

where E[N] is the expected number of jobs in the system, λ is the average arrival rate into the system, and E[T] is the mean time jobs spend in the system



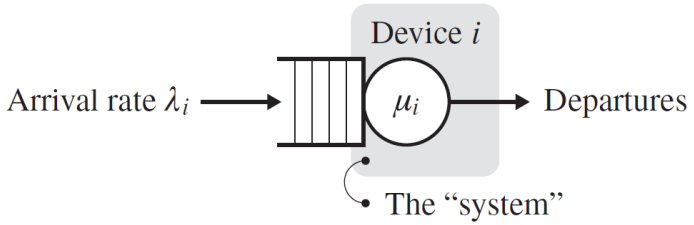
- It is important to note that Little's Law makes **no assumptions** about the arrival process, the service time distributions at the servers, the network topology, the service order, or anything!

Little's Law

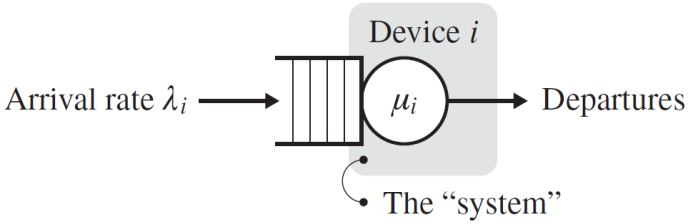
- The utilization law says

$$\rho_i = \lambda_i \cdot \frac{1}{\mu_i}.$$

- Do you see how to use Little's Law to prove this? What should we define the "system" to be?
  - Let the "system" consist of just the service facility without the associated queue, as shown in the shaded box of the following figure. Now the number of jobs in the "system" is always just 0 or 1.



Little's Law



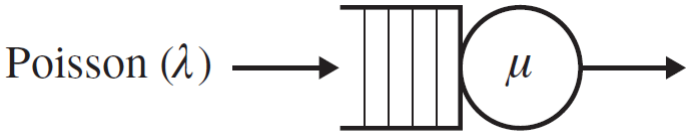
- **Question:** What is the expected number of jobs in the system as we have defined it?
  - Observe that, $\boldsymbol{\rho_i}$ represents both the long-run fraction of time that device ***i*** is busy and also the **probability** that device ***i*** is busy
  - The number of jobs in the system is 1 when the device is busy (this happens with probability $\boldsymbol{\rho_i}$) and is 0 when the device is idle (this happens with probability 1- $\boldsymbol{\rho_i}$). Hence, the expected number of jobs in the system is $\boldsymbol{\rho_i}$

  - So, applying Little's Law, we have

$$\begin{aligned}\rho_i &= \text{Expected number jobs in service facility for device } i \\ &= (\text{Arrival rate into service facility}) \cdot (\text{Mean time in service facility}) \\ &= \lambda_i \cdot \mathbf{E}\left[\text{Service time at device } i\right] \\ &= \lambda_i \cdot \frac{1}{\mu_i}.\end{aligned}$$

M/M/1 Queue

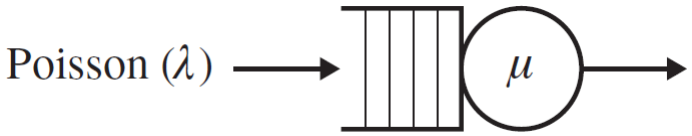
- The simplest queueing model consists of a single server and a single queue
  - The service times follow an Exponential distribution with mean $1/\mu$
  - The customers (jobs) arrive into the system according to a Poisson process with rate λ
  - Such a system is referred to as an M/M/1 queueing system, as shown below



- M/M/1 is Kendall notation and describes the queueing system architecture
  - The "M" in this first slot stands for "memoryless" and says that the interarrival times are Exponentially distributed
  - The second slot characterizes the distribution of the service times
  - The third slot indicates the number of servers in the system
  - A fourth slot is typically used to indicate an upper bound on the capacity of the system. Sometimes, however, the fourth slot is used to indicate the scheduling discipline

M/M/1 Queue

- For an M/M/1 queue, we would like to know
  - the probability that the system has n jobs, where n = 0, 1, …
  - the expected number of jobs in the system
  - the expected time in the system and queue

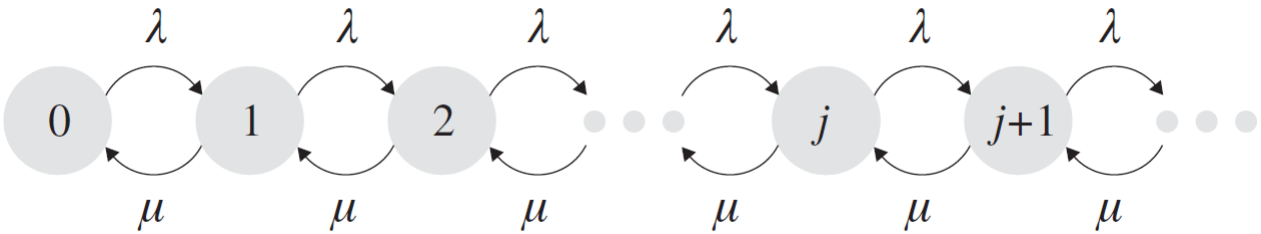


M/M/1 Queue

- To obtain the probability distribution of the number of jobs in the system, we employ a continuous time Markov chain (CTMC) to represent the M/M/1 queue

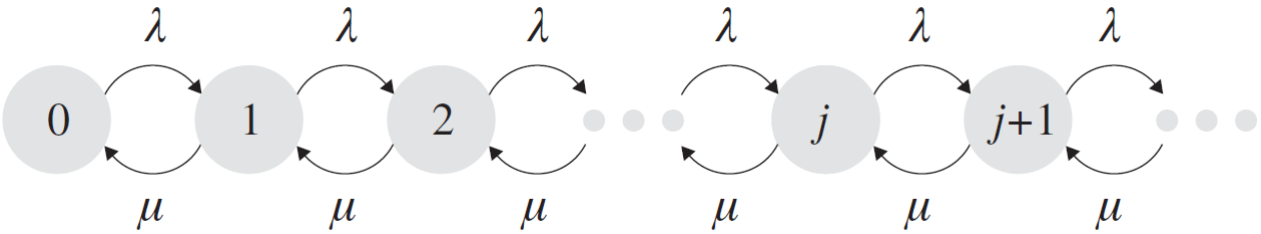
- CTMC is a powerful tool for modeling continuous-time stochastic systems. However, the formal definition of CTMC needs ergodic theorem, which is far beyond the scope of this class

- We only show how to use the CTMC to analyze the M/M/1 queue



- The numbers in the CTMC show the number of jobs in the system

M/M/1 Queue



- Let π_i denote the probability that the M/M/1 system has i jobs, where i=0, …
- We can obtain π_i by solving the so-called **balance equations**
- The balance equations equate the rate at which the system leaves state i with the rate at which the system enters state i

State	Balance equation	Simplified equation
$0:$	$\pi_0\lambda = \pi_1\mu$	$\Rightarrow \pi_1 = \frac{\lambda}{\mu}\pi_0$
$1:$	$\pi_1(\lambda+\mu) = \pi_0\lambda + \pi_2\mu$	$\Rightarrow \pi_2 = \left(\frac{\lambda}{\mu}\right)^2 \pi_0$
$2:$	$\pi_2(\lambda+\mu) = \pi_1\lambda + \pi_3\mu$	$\Rightarrow \pi_3 = \left(\frac{\lambda}{\mu}\right)^3 \pi_0$
$i:$	$\pi_i(\lambda+\mu) = \pi_{i-1}\lambda + \pi_{i+1}\mu$	$\Rightarrow$???

$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \pi_0.$$

M/M/1 Queue

- Note that the sum of the probabilities equals 1, i.e.,

$$\sum_{i=0}^{\infty} \pi_i = 1$$

$$\Rightarrow \sum_{i=0}^{\infty} \left(\frac{\lambda}{\mu}\right)^i \pi_0 = 1$$

$$\Rightarrow \pi_0 \left(\frac{1}{1-\frac{\lambda}{\mu}}\right) = 1$$

$$\Rightarrow \pi_0 = 1 - \frac{\lambda}{\mu}$$

- Therefore, substituting back into the equation for π_i, we obtain

$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \left(1 - \frac{\lambda}{\mu}\right) = \rho^i (1-\rho)$$

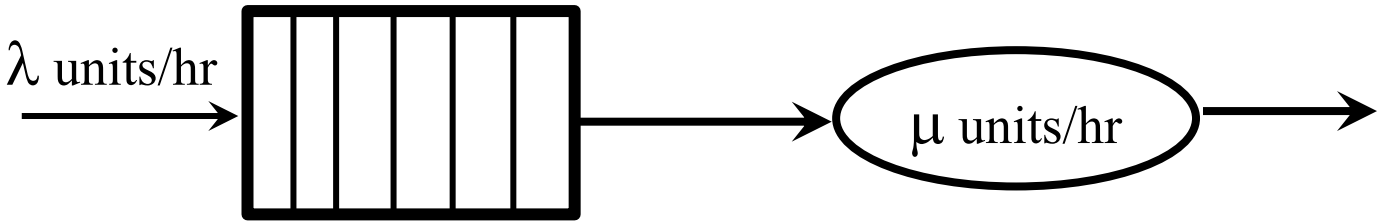
$$\pi_0 = 1 - \frac{\lambda}{\mu} = 1 - \rho$$

M/M/1 Queue

- The expected number of customers in the system can be derived as follows:

$$
\begin{aligned}
\mathbf{E}[N] &= \sum_{i=0}^{\infty} i\pi_i \\
&= \sum_{i=1}^{\infty} i\pi_i \\
&= \sum_{i=1}^{\infty} i\rho^i(1-\rho) \\
&= \rho(1-\rho)\sum_{i=1}^{\infty} i\rho^{i-1} \\
&= \rho(1-\rho)\frac{d}{d\rho}\left[\sum_{i=0}^{\infty}\rho^i\right] \\
&= \rho(1-\rho)\frac{d}{d\rho}\left[\frac{1}{1-\rho}\right] \\
&= \rho(1-\rho)\frac{1}{(1-\rho)^2} \\
&= \frac{\rho}{1-\rho}.
\end{aligned}
$$

Little's Law in M/M/1



Avg. number waiting $E[\mathbf{N_Q}]$ + Avg. number in server $\boldsymbol{\rho}$ = Avg. number in system $E[\mathbf{N}]$

Avg. waiting time $E[\mathbf{T_Q}]$ + Avg. service time $E[\mathbf{S}]$ = Avg. time in system $E[\mathbf{T}]$

Little's Law:

$$E[\mathbf{N_Q}] = \lambda E[\mathbf{T_Q}] \qquad \boldsymbol{\rho} = \lambda E[\mathbf{S}] \qquad E[\mathbf{N}] = \lambda E[\mathbf{T}]$$

PASTA (Poisson Arrivals See Time Averages)

- Mercurio's is a pizza restaurant in Pittsburgh. Customers arrive randomly (like when you're hungry and decide to grab a slice). The restaurant has a single counter, and sometimes there's a line, sometimes not
- **Time Average**: Suppose you check the pizza counter every minute for a whole day and note how many people are in line. The average number of people you'd calculate over the day is the **time average**
  - Example: If there's no line half the time and 4 people in line the other half, the time average is 2
- **Arrival Average:** Now imagine customers walking into the restaurant. Each customer sees the line at the exact moment they arrive.
  - If you asked every arriving customer, "How many people were in line when you walked in?" and averaged their answers, that's the arrival average.
- **PASTA:** If customers arrive in a "totally random" way (like a Poisson process), the **average line length** they see when they arrive equals the **time average**
  - **Random arrivals don't "miss" busy or quiet times.** They "sample" the line in a way that perfectly matches the system's long-term average.
  - Example: If people only arrived during rush hour, they'd always see a long line. Their arrival average would be higher than the time average (which includes quiet times).

PASTA (Poisson Arrivals See Time Averages)

- PASTA helps designers answer questions like:
  - "If we add a second counter, how much shorter will the line **feel to customers**?"
  - "What's the chance **a customer walks in and sees no line**?"
- By knowing the time average, we automatically know what arriving customers experience—no extra math needed!

Practice

- The Bijou Theater in Hermosa Beach, California, shows vintage movies. Customers' average inter-arrival time is 12 seconds. The ticket seller averages 9 seconds per customer. Assume Poisson arrival and exponential service time, how long is the waiting line (exclude the one being served) on average?

Practice

- Given an M/M/1 queue, what is the maximum allowable arrival rate of jobs if the mean job size (service demand) is 3 minutes and the mean waiting time (E[T_Q]) must be kept under 6 minutes?

Practice

- Given an M/M/1 system (with $\lambda < \mu$), suppose that we increase the arrival rate λ and the service rate μ by a factor of k each.

- Question: How are the following affected?
  - utilization, ρ?
  - throughput, X?
  - mean number in the system, E[N]?
  - mean time in system, E[T]?

Practice

- Suppose ***m*** independent Poisson packet streams, each with an arrival rate of ***λ/m*** packets per second, are transmitted over a communication line. The transmission time for each packet is Exponentially distributed with mean ***1/μ***. We wish to analyze the performance of two different approaches to multiplexing the use of the communication line, and figure out how the two methods compare with respect to mean response time?
  - Statistical Multiplexing (SM): This approach merges the ***m*** streams into a single stream. Because the merge of ***m*** Poisson streams is still a Poisson stream, the resulting system may be modeled as a simple M/M/1 queueing system as shown
  - Frequency-Division Multiplexing (FDM): This approach leaves the ***m*** streams separated and divides the transmission capacity into ***m*** equal portions as shown